

오늘도 장비 대여

Today's Weapon Rental — 길드 경영 시뮬레이션 · Unity / C# · 1인 개발 · Google Play 정식 출시

모험가에게 던전을 주선하고 무기를 대여해 수수료를 받는 경영 게임입니다. 하루가 아침·낮·저녁·밤으로 흐르고, 플레이어는 정보(점술·수색·대화)를 사서 성공률을 올리며 주간 의뢰와 벌금 압박 속에서 길드를 키웁니다. 기획·프로그래밍·서버·운영 툴·출시까지 전 과정을 혼자 진행했습니다.

이름 / 연락처	[이름] · dldudcjs12345@gmail.com
기간	2026.03 ~ 2026.08 (5개월, 커밋 1,374개)
역할 · 기여도	1인 개발 — 기획 · 클라이언트 · 서버(Cloud Code) · 운영 툴 · 밸런싱 · 출시 (100%)
플랫폼	Android — Google Play 정식 출시 v1.0.0 (2026-08-17), 알파 3회 선행
스토어	play.google.com/store/apps/details?id=com.IdeaBank.TodaysWeaponRental
GitHub	github.com/seunghee-2002/Todays-Weapon-Rental-code (자체 작성 코드·문서만 담은 열람용 저장소)
소개 페이지	seunghee-2002.github.io/todays-weapon-rental

**75,415**

자체 C# 라인 (307 파일, 에셋 코드 제외)

36 / 56 / 25

Manager / View / Controller 클래스

35건

시뮬레이션 실측 근거를 첨부한 밸런스 변경 기록

4개 언어

ko · en · ja · zh-Hans, String Table 2,110키

핵심 성과 요약

- **헤드리스 밸런스 시뮬레이터**를 만들어 밸런싱을 "감"에서 "실측"으로 바꿨습니다 — Unity batchmode로 100시드 × 100주 × 3페르소나를 돌리고, 모든 가격을 "모험 1회 수입(R)의 배수"로 읽는 기준 체계를 세웠습니다. 밸런스 변경 35건이 전부 before/after 수치를 갖습니다. (§3)
- **서버 검증 기반 라이브옵스** — Unity Cloud Code 함수 11개로 리더보드 점수(4단계 검증·재사용 차단·TOCTOU 처리), 닉네임, 밴, 기기 이전 코드를 서버에서 판정하고, 세이프 동기화는 서버 시각 스탬프로 충돌을 해소합니다. FastAPI 어드민 사이트와 GitHub Actions 자동 배포까지 포함합니다. (§4)
- **기획 데이터·다국어 파이프라인** — SO↔CSV diff/검증 툴로 "무음 데이터 파손"을 막고, 폰트 폴백 체인과 실측 기반 아틀라스 판단으로 프리팹 1,109참조를 수정 없이 4개 언어로 전환했습니다. (§5)
- **출시와 운영** — 알파 0.0.1→0.0.3→정식 1.0.0. 플레이테스트 지적 4건을 시스템 확장으로 대응했고, 애널리틱스 이벤트 28종을 CI가 코드와 대조합니다. (§7)

기술 스택

엔진 · 언어	Unity 2022.3 LTS · C# · uGUI · TextMeshPro · Addressables · Spine 런타임(외형 애니메이션) · ScriptableObject 데이터
온라인	Unity Gaming Services — Authentication(익명) · Cloud Save(Default/Protected) · Cloud Code(JS 11개) · Leaderboards · Analytics
다국어	Unity Localization 1.5 — String Table 7컬렉션 · 로케일별 폰트 폴백 · PowerShell 감사 스크립트
툴 · 운영	Python 3 (FastAPI · httpx · pytest · Playwright) · Google OAuth · Docker · GitHub Actions · PowerShell · Unity Editor 확장 15종

§1 게임 개요

§2 아키텍처와 저장 계층

§3 밸런스 시뮬레이터 ★

§4 UGS 라이브옵스 ★

§5 데이터·다국어 파이프라인

§6 에디터 툴과 CI

§7 출시와 운영

§8 한계와 회고

§1 게임 개요

코어 루프 — 하루 = 정보를 사서 위험을 줄이는 경영

시간대	플레이어가 하는 일
아침 06-09	이벤트 NPC(대장장이·무기상·수수께끼 상자 등) 응대, 09:00 의뢰판 갱신
낮 09-18	방문한 모험가에게 던전 배정 + 무기 대여. 점술·수색·대화로 숨은 스탯/특성을 공개하고 성공률을 올림
저녁 18-21	전령 보고 — 모험 결과 정산(수수료 7% + 대여료 10% ± 특성), 재료 회수
밤 21:00	자동 일시정지 → 다음 날. 주말엔 주간 의뢰 판정, 미달 시 벌금

실시간 1초 = 게임 3분, 배속 조절 가능. 게임오버(파산)되면 회차가 끝나고 유산(영구 업그레이드)이 남아 다음 회차가 조금 쉬워지는 로그라이트 구조입니다. 40주 캠페인 이후는 난이도 등급이 붙은 엔드리스 의뢰가 이어집니다.

1.0 콘텐츠 규모

- 무기 178종 · 던전 28종 · 재료 96종 · 액티브 아이템 30종 · 주간 의뢰 56종 · 던전 이벤트 97종
- 모험가 페르소나 20종 + 네임드 8명, 특성 16종, 아침 이벤트 NPC 9종
- 유산 업그레이드 20종, 대장간(강화·진화·재부여·분해·제작)
- 리더보드(일수 기준), 닉네임, 기기 이전, 계정 초기화, 강제 업데이트



매일 바뀌는 던전 3개 선택



모험가 방문 — 정보 공개 후 배정



정산 — 수수료 구조가 그대로 보임

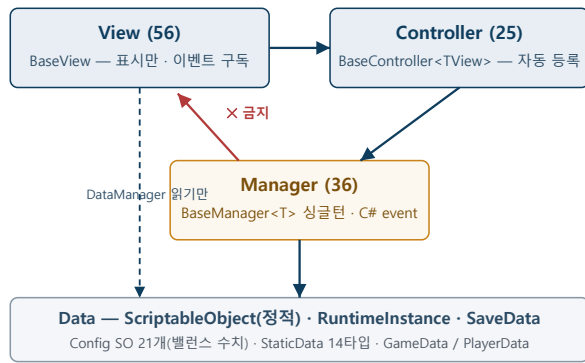


대장간 강화 — 부가효과 5종

설계의 중심 질문 — "정보를 사는 비용이 성공률 상승으로 회수되는가?" 이 게임의 모든 골드는 모험 성공에서 나오므로, 점술·수색·대화·무기 가격을 전부 **모험 1회 순수입(R)** 단위로 정의했습니다(§3-3). 상급 플레이어는 정보를 써서 R 자체가 커지고(약 1,100G), 초급은 정보를 안 써서 R이 작습니다(약 600G) — 실력 격차가 가격이 아니라 R에 녹아 있게 만든 것이 밸런스 설계의 핵심입니다.

§2 아키텍처와 저장 계층

4계층 구조 — "Manager는 View를 모른다"



호출	규칙
View → Controller	도메인 매니저는 반드시 Controller 경유
View → DataManager	정적 SO 카탈로그, 읽기 전용만 허용
View → AnalyticsManager	관측 전용(상태 미변경)이라 직통 허용 — 컨트롤러 없는 View의 <code>btn_clicked</code>
Manager → View/Controller	절대 금지. Manager는 <code>event Action</code> 만 노출

거대 매니저는 기능별 partial로 분할 — `AdventureManager` 7파일 3,127줄(Calculations · EventProcessor · Sequence · HeraldReport · Mood), `VisitorManager` 5파일, `Insight` 5파일.

데이터 접근 원칙 — "ID로 굽고, 참조로 복원"

`RuntimeInstance`는 정적 SO를 참조로 직접 들고, 저장 시에만 ID 문자열로 굽습니다. `RuntimeInstance`는 `DataManager`를 모르며, 복원은 각 도메인 매니저의 `Initialize(GameData)`가 맡습니다.

```
// 굽기 - Utils/SaveDataConverter.cs
weaponDataID = instance.weaponData.StaticID,
// SO 참조 → ID
dungeonDataID = instance.dungeon?.StaticID ?? "unknown",
rentedWeaponInstanceID = instance.rentedWeapon?.instanceID,

// 복원 - Systems/InventoryManager.cs:32 LoadFromGameData
var weaponData = DataManager.Instance.GetWeapon(saveData.weaponDataID);
if (weaponData == null) continue;
// 폐기된 ID는 스킵
var effects = ReifyEffectsFromSave(saveData.effects);
ownedWeapons.Add(new WeaponInstance(weaponData, effects, saveData));
```

`DataManager`의 판정 기준: "이 메시드가 게임 규칙을 아는가(→ 도메인 매니저), 필드 값만 보는가(→ `DataManager`)". 예: `GetEnforceMaterialByGrade`는 규칙이라 `BlacksmithManager`로.

초기화 순서의 단일 진입점

`GameManager([DefaultExecutionOrder(-100)])`의 `InitializeManagers()`가 21개 매니저를 의존 순서로 초기화합니다: Inventory → ... → Visitor → Adventure(모험가·무기가 먼저 있어야 진행 중 모험 복원) → ... → Tutorial(맨 끝).

저장 계층 규약 3가지

문서 원문: "데이터 손실 버그가 반복적으로 나왔던 지점이라 공통 장치로 고정해 두었다."

규약	구현
원자적 쓰기	<code>SaveManager.WriteAllTextAtomic</code> — <code>.tmp</code> 에 쓰고 <code>File.Replace</code> , 직전본은 <code>.bak</code> . 로드 시 본 파일 손상 → <code>.bak</code> 복구, 손상본은 <code>.corrupt</code> -타임스탬프로 보존
확정 직후 저장	<code>SaveAfterCommittedAction(reason)</code> — 강화·상자·모험 출발 등 도박성 결과 확정 직후 저장해 강제 종료 세이브스کم 차단
로드 후 sanity pass	<code>RepairLoadedGameData()</code> — 복원 실패로 드롭된 모험의 <code>isRented</code> / <code>isAdventuring</code> 해제, 고아 아이템 반환

유산(PlayerData) 로드는 `LoadDataStatus` (Success / Missing / RestoredFromBackup / Corrupted)를 함께 반환 — Corrupted면 빈 데이터로 조용히 확정하지 않고 클라우드 복구·안내를 거칩니다.

코드: `Scripts/Core/GameManager.cs#L46, #L123, #L260` · `Scripts/Systems/SaveManager.cs#L195` · `Scripts/Systems/InventoryManager.cs#L32` · `Scripts/Utils/SaveDataConverter.cs`

문제

"무기 3,000G가 비싼지 싼지 판단할 근거가 없어, 조정할 때마다 임의 수치를 골랐다." 목표를 '생존 주차' 같은 종속변수에 걸어두니 A를 고치면 B가 깨지는 두더지 잡기가 됐습니다.

접근

게임 로직을 헤드리스로 돌리는 시뮬레이터(5,026줄)를 만들고, 모든 가격을 "모험 1회 순수입 R의 배수"로 읽는 기준 체계를 세웠습니다. 수치는 100% 실제 SO 에셋에서 읽습니다.

결과

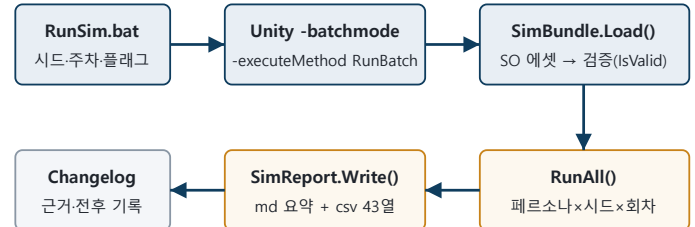
페르소나 3층 통과율 96 / 70 / 33%로 레벨디자인 합격선(95↑/70/40↓) 달성. 밸런스 변경 35건 전부 before/after 리포트 첨부. 리포트 255개 생성.

3-1. 설계 — 미리 로직 + 실제 에셋 + 결정론

- 수치는 에셋, 공식은 미리. `SimBundle.Load()` 가 Config SO 15종과 던전·무기·의뢰·모험가 데이터를 `AssetDatabase`로 통째로 읽습니다. 그래서 `PriceTierConfig` 값을 바꾸면 시뮬 코드는 손댈 필요가 없습니다. 런타임 `MonoBehaviour` 매니저는 호출하지 않고 계산식만 출처 주석을 달아 재구현했으며, 순수 static 유틸(`StatCurve`, `TypeAdvantage`)은 그대로 공유합니다.
- 페르소나 3종(상급/중급/초급) — 차이는 정보 사용량 하나뿐. 판정은 `SimWorld`가 실제 수식으로 하고 페르소나는 선택만 합니다.
- RNG 스트림 분리 — 메인/특성/점술/네임드/엔드리스 5개 시드를 따로 둡니다. 새 시스템을 미리링할 때 전용 rng를 쓰므로 그 축을 OFF로 끈 런이 도입 이전 코드와 비트 단위로 동일합니다. 이걸로 "새 시스템이 결과를 바꿨는가"를 무결성 A/B로 검증합니다.
- 회차 반복 — 폐업 시 유산 획득 → 다음 회차(시드당 최대 5회). 유산 20종 단독 효과 스위치는 같은 시드끼리 짝지은 차이의 중앙값(`paired median`)을 씁니다 — 독립 비교는 시드 경로 발산만으로 10% 흔들려 효과가 묻히기 때문입니다.

```
// SimCore.cs:700-704 - 축별 독립 RNG 스트림
rng = new Random(seed*977 + persona.Name[0] + (runIndex-1)*7919);
traitRng = new Random(seed*733 + ...); // 특성 룰
seerRng = new Random(seed*509 + ...); // 점술
namedRng = new Random(seed*331 + ...); // 네임드 스폰/재방문
questRng = new Random(seed*199 + ...); // 엔드리스 템플릿 추천
```

3-2. 실행 파이프라인



실행기	기본값	산출
RunSim.bat	100시드 / 100주 / 5회차	페르소나 비교 · R 비율표 · 회차 곡선
RunSweep.bat	100 / 70	유산 20종 단독 효과 (21구성)
RunEndlessSweep.bat	60 / 100	엔드리스 템플릿 16종 단독 반복 (17구성)

진단 플래그 `-simForceMorning` / `-simForceReroll`: "이득일 때만 한다"는 봇 정책은 지출 0을 만드는데, 그 0이 봇 결함인지 콘텐츠가 죽은 것인지는 강제 참여로 한 번 더 돌려야 구분됩니다 — 실제로 4번 겪은 함정을 체크리스트로 남겼습니다.

3-3. R 기준 체계 — 가격을 "모험 몇 회분"으로 읽기

R = 모험 1회 성공 시 순수입. 이 게임의 모든 골드는 모험에서 나오므로 모든 가격을 R의 배수로 정의하고, 조정 위치를 고정했습니다.

- 1차 R 확정 던전 보상 × 수수료 - 다른 모든 것의 기준점
- 2차 가격 = R 배수 무기 1.8R~30R / 수색 0.3R / 점술 0.6R / 재부여 ~6R
- 3차 압박 = 수입 비율 벌금 (1200 + 1300w)
- 4차 페르소나 격차 생존 주차는 조정 대상이 아니라 "검증 대상"

부수 효과: R이 실력 지표를 겸합니다 — 상급자 R≈1,100G, 중급 ≈700G. 상성을 맞춰 팁 수수료를 받기 때문에 정보의 가치가 R에 이미 녹아 있습니다.

코드: `Scripts/Editor/BalanceSimulator/SimCore.cs#L695` (`SimWorld.Run`) · `#L1666` (`LaunchAdventure` 미리) · `SimReport.cs#L500` (`AppendRatioTable`) · `#L76` (`paired median`) · `BalanceSimulatorWindow.cs#L164` (`RunBatch`) · `Tools/BalanceSim/`

3-4. 최종 상태

(sim_summary_20260801_201317 · 100시드 × 40주 × 5회차)

지표	상급자	중급자	초급자	합격선
퀘스트 통과율	96%	70%	33%	95 ↑ / 70 / 40 ↓
회차 수명	40주	31주	19주	—
폐업률	10%	93%	100%	—
최고 평판	Diamond	Diamond	Bronze	—
주간 순수입	26,851G	22,982G	5,760G	—
주간 수색 지출	10,825G	0G	0G	—

R 기준 비율표 (상급자)

구간	R	무기	제작	재부여	대장간	수색	점술	벌금/수입
1~10주	757G	1.8R	1.1R	5.9R	3.3R	0.3R	0.4R	0%
11~20주	951G	3.7R	1.1R	5.9R	9.0R	0.3R	0.3R	0%
21~30주	1,086G	4.2R	1.2R	6.2R	13.9R	0.2R	0.3R	1%
31~40주	1,161G	4.0R	1.1R	5.8R	16.4R	0.2R	0.3R	6%

같은 파일의 초급자는 R 600 → 1,376G, 벌금/수입 33% → 117% → 86% — 압박이 "수입 비율"로 설계된 대로 초급자에게만 걸립니다.

3-5. 결정 사례 ② 무기 제작비 계단 4.5배 → 2.0배 (2026-08-01)

근거 상점가는 고정인데 제작 골드에만 주차 계단(1→4.5배)이 붙어 후반에 제작이 구매보다 비싸졌습니다. 레시피 24종 전수 계산: 41주+ 영웅 104~120%, 전설 116~123%. "재료를 모으는 수고를 더 하고 돈도 더 내는 구조."

수정 PriceTierConfig.weaponCraftCost 1/1.5/2.5/3.5/4.5 → 1/1.2/1.5/1.8/2.0 (대장간·재부여와 동일 곡선으로 통일)

구간 (상급자)	구매 전→후	제작 전→후
11~20주	3.9R → 3.7R	1.8R → 1.1R
31~40주	2.9R → 4.0R	1.6R → 1.1R

우려 2건도 실측으로 해소 — 후반 골드 축적 폭주 없음(40주 보유 골드 1,684,102 → 1,686,848G), 제작이 구매를 잠식하지 않음(구매 지출 동일, 보유 무기 27 → 34자루).

문서: Docs/오늘도장비대여/Balance/Reference/밸런스_시뮬레이터_정리.md · Balance/Changelog/2026-07-29_엔드리스_난이도등급과_벌금곡선.md · 2026-08-01_무기제작비_계단_완화.md · Docs/Simulation/

3-5. 결정 사례 ① 엔드리스 난이도 등급 신설 (2026-07-29)

발견 엔드리스 템플릿 16종은 아키타입(휴식/표준/소피크/시험)으로 분류돼 있었지만 실측 통과율은 분류와 무관하고 심지어 역전 — "'쉬움'으로 구운 W61(36.8%)이 '최고난도' W70(96.7%)보다 2.6배 어려웠다." 난이도를 만드는 건 아키타입이 아니라 병목 3종(저가용 던전 요구 / 희소 무기 석궁 요구 / 돌의 조합)이었습니다.

수정 QuestDifficulty 4단계(Easy/Normal/Hard/Extreme)를 병목 개수 기반으로 신설, 61주+ 벌금을 고정 79,200G에서 79,200 + 8,000 × (주차-61) 곡선으로 전환.

스텝 before → after (상급자, 60시드 × 17구성)

템플릿	통과율	폐업률	주간 순수입
전 (sim_endless_sweep_20260729_224539) — 16종 전부 "Easy", 44~46%로 뭉개짐			
기준선	44.2%	66.1%	-79,002G
W41 [Easy]	46.2%	64.7%	-76,144G
후 (..._230337, 18분 뒤) — 라벨 순서 = 통과율 순서			
W41 [Easy] 단골들의 주문	97.6%	0.0%	+104,853G
W45 [Normal] 끝나지 않는 명성	94.6%	0.0%	+93,683G
W49 [Hard] 쏟아지는 주문	74.6%	29.4%	+23,724G
W53 [Extreme] 살아있는 전설	46.7%	60.5%	-57,502G

벌금 기울기 A/B가 없었다면 틀린 값을 골랐을 것 — 기울기 4,000에서는 상급자 폐업률 31% vs 중급자 21%로 페르소나 순서가 역전됐습니다(재투자로 현금이 얇은 상급자가 먼저 죽음). 8,000에서 70% vs 83%로 정상화. 직관으로는 4,000을 골랐을 겁니다.

3-6. 정직하게 적어야 할 한계

- 미러 로직의 이중 유지보수 — 값은 에셋에서 읽지만 공식이 바뀌면 손으로 맞춰야 합니다. 캠페인 길이를 상수로 뒀다가 조용히 어긋난 적이 있어 EndlessQuestConfig에서 읽도록 고쳤습니다.
- 봇 정책이 결과를 지배 — "중급이 초급보다 순수입이 낮은 것은 봇이 무기를 2자루씩 사는 정책 탓일 수 있다." 서열은 믿고 절대 값은 참고만 합니다.
- 미모델링 — 기분 배율, 대화, 단골, 네임드 부활. 수치로 못 고치는 구조 문제 4건(황금 아물렛, 인벤토리 슬롯, 네임드 등장률, 유산 획득 구조)은 로드맵에 남겼습니다.

문제

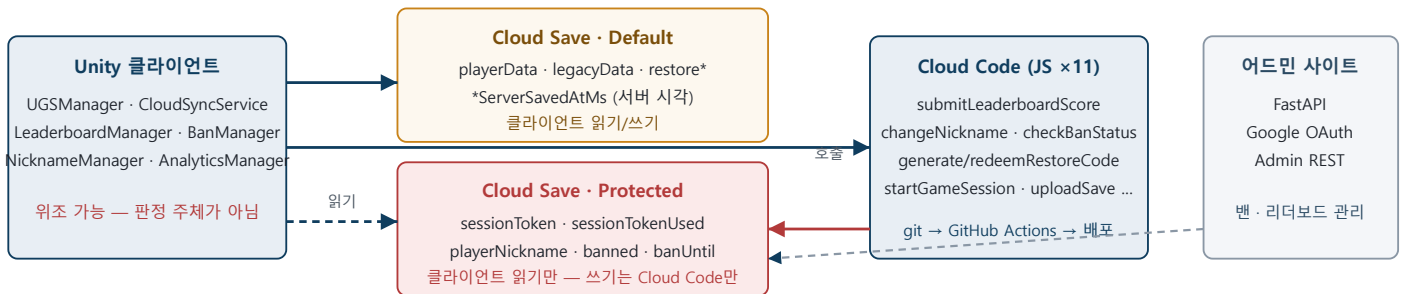
1인 개발 모바일 게임에 리더보드·닉네임·기기 이전·밴이 필요했지만 전용 서버를 운영할 여력은 없었습니다. 클라이언트가 쓰는 값은 전부 위조 가능하다는 전제로 시작했습니다.

접근

Cloud Save의 **Protected access class** (클라 읽기만, Cloud Code만 쓰기)를 신뢰 경계로 삼고, 판정이 필요한 모든 쓰기를 Cloud Code 11개 함수로 옮겼습니다. 서버 함수는 git이 단일 원본이고 CI가 배포합니다.

결과

리더보드 4단계 검증, 서버 스탬프 세이브 동기화, 24h 복구 코드, 밴 3중 검사, 어드민 사이트(pytest 273줄)까지 DB 없이 운영. 정식 출시 후 운영 중.



4-1. 리더보드 점수 제출 — 4단계 서버 검증

1. 타입/범위 — 양의 정수, ≤ 9999
2. 밴 검사 — Protected의 banned / banUntil 재확인 (에러 2006)
3. 세션 토큰 대조 + 재사용 차단 — startGameSession 이 서버에서 만든 토큰을 Protected에 저장, 제출 시 대조하고 sessionTokenUsed 로 Replay 차단. 사용 처리는 읽어들 writeLock 으로 조건부 갱신 → 동시 제출(TOCTOU) 시 한쪽만 성공
4. 시간 검증 — 세션 경과 시간 $\geq \text{claimedDays} \times 30\text{초}$

"검증하지 않기로 한" 결정도 코드에 남겼습니다

(submitLeaderboardScore.js:125-148). claimedDays 와 클라우드 playerData.currentDay 를 대조해 거부하던 로직을 로그만 남기도록 바꿨습니다. 둘 다 클라이언트가 쓰는 값이라 치터는 저장본을 먼저 위조하면 통과하는 반면, 정상 유저는 일자 경계에서 업로드가 한 번만 늦어도 거부당해 기록을 영구히 잃습니다. "막지 못하는 치터 vs 잃는 정상 유저"를 저울질한 판단입니다.

4-2. 닉네임 — 서버 검증 풀세트

길이 2~12 · 정규식 `^[가-힣a-zA-Z0-9_]+$` · 욕설 필터는 NFKC 정규화 + 소문자화 후 매칭(전각/호환문자 우회 차단), 금칙어는 SO에서 스크립트로 JS에 자동 인라인. 변경 횟수는 writeLock 조건부 증가로 동시 요청 회피 차단. 리더보드 표시 직전 서버가 < > 를 제거해 TMP rich-text 주입도 막습니다.

4-3. 세이브 동기화 — 시계는 서버 것만 믿는다

uploadSave 가 저장과 동시에 서버 시각 스탬프를 기록하고, 클라이언트는 자기 마지막 업로드 스탬프만 기억합니다. 기기 시간 조작으로 오래된 저장이 최신으로 오판되는 일이 없습니다.

```

// CloudSyncService.ResolveGameDirection 요지
클라우드 스탬프 > 이 기기의 마지막 업로드 스탬프 → 클라우드 채택
같다 (= 클라우드가 이 기기가 올린 분) → 로컬 채택
(오프라인 진행 가능성)
로컬 legacy 손상(IsLegacyCorrupted) → 스탬프 무시, 무조건 클라우드
    
```

- legacy는 독립 동기화 — 게임오버 직후 새 기기처럼 playerData 가 없어도 유산은 내려받아야 하므로.
- 게임오버 = 클라우드 비우기 — 안 하면 메인메뉴 동기화에서 죽은 회차가 부활. 실패 시 pendingCloudClear 로 재시도하며, 이 정리는 항상 Download/UploadPending보다 먼저 호출됩니다(3곳 모두 주석으로 고정).
- 이중 타임아웃 게이트 — UGS 초기화 10초 + 절차 10초. 상한 뒤 늦게 온 응답은 폐기(사용자가 이미 게임에 들어갔을 수 있으므로 로컬을 덮어쓰지 않음). 로그인 실패는 SignInAttemptFinished 신호로 조기 탈출.

4-4. 기기 이전 · 밴

- 복구 코드 — "playerId-secret", 24h 만료, 1회용. secret 불일치와 데이터 없음을 같은 에러코드로 응답해 계정 존재 여부 오라클을 차단. 원 기기는 30초 폴링으로 사용 감지 후 전체 wipe.
- 밴 — source of truth는 Protected 3기. 검사는 앱 실행 · 재연결 · 점수 제출(실질 차단점) 3중. SDK가 에러코드 2006을 9009로 뭉개는 경우가 있어 메시지 문자열에서 JSON을 정규식 추출하는 우회까지 넣었습니다.
- 어드민 사이트 — DB 없는 stateless MVP: 어드민이 죽어도 Unity Dashboard에서 같은 키를 직접 조작 가능. Google OAuth + 이메일 화이트리스트(비면 fail-closed), 502 변환 시 detail 축소로 시크릿 누출 방지. pytest로 밴 서비스·인증 경계 커버.

코드: Tools/CloudCode/submitLeaderboardScore.js#L41 · changeNickname.js#L46 · redeemRestoreCode.js#L69 · Scripts/Systems/CloudSyncService.cs#L76 · Scripts/Systems/LeaderboardManager.cs#L378 · Tools/admin-site/app/services/ban_service.py#L48

4-5. "조용히 사라지는 데이터"를 CI로 막기

UGS Analytics는 **미등록 이벤트/파라미터를 invalid로 조용히 버립니다**. 코드와 대시보드 스키마가 어긋나면 지표가 소리 없이 빠집니다. 그래서 `events.json` (이벤트 28개 · 파라미터 73개)을 단일 원본으로 두고 세 방향으로 대조합니다.

도구	하는 일
<code>analytics.py check</code> (530줄, 표준 라이브러리만)	C# 소스를 정적 파싱해 <code>Send("event", dict)</code> 호출의 이벤트 명·딕셔너리 키를 추출하고 <code>events.json</code> 과 대조. 주석 제거·괄호 매칭·제네릭 제거· 변수로 넘긴 딕셔너리를 메서드 범위에서 역추적 . 어긋나면 <code>exit 1</code> — GitHub Actions에서 실행
<code>analytics.py render</code>	<code>events.json</code> → 대시보드 등록 문서의 생성 구간 갱신, CI가 diff 확인
<code>dashboard_sync.py</code> (626줄, Playwright)	Unity가 이벤트 스키마 공개 API를 제공하지 않아(조사 근거를 docstring에 기록) 대시보드를 브라우저로 조작. <code>pull / diff / apply</code> , 삭제는 하지 않음. 비공식 경로임을 스스로 명시

클라이언트 쪽 설계 포인트

- 동의 게이트 — "물어봤는가"와 "철회했는가"를 **별개 키**로 (하나로는 '동의함'과 '아직 안 물어봄'을 구분 못 함). 계정 초기화에도 유지되는 기기 단위 설정.
- `required` 를 어디에도 두지 않음 — 조건부 파라미터가 많아 하나라도 빠지면 이벤트 전체가 폐기되므로.
- 에디터에서는 전송하지 않고 로그만 — 실지표 오염 방지. 게임오버처럼 앱이 곧 꺼지는 시점만 `Flush()`.
- 4단계 스키마: 게임 흐름 / 패널 열림·닫힘(`duration_sec`) / 버튼 / 도메인 결과. 분석 목표 G1~G27과 퍼널 A~L에 대응하는 Snowflake SQL 템플릿 786줄 문서화.

4-6. 배포 — 서버 함수도 git이 단일 원본

`deploy.py` (593줄, 표준 라이브러리만): `envs / diff` (차이 있으면 `exit 1`) / `push --yes`. `module.params` 블록을 파싱해 Admin API 타입으로 변환, 5xx/429 재시도, **대시보드에서 직접 편집한 흔적 (publish 미완 초안) 탐지**. GitHub Actions가 `main` 푸시 시 `diff` → `push` → `diff` 로 배포하고, 대시보드 수동 편집은 다음 배포 때 되돌립니다.

코드: [Tools/Analytics/analytics.py#L240 \(scan_file\)](#) · [#L196 \(resolve_variable_keys\)](#) · [Tools/Analytics/events.json](#) · [Tools/CloudCode/deploy.py](#) · [.github/workflows/](#) · [Scripts/Systems/AnalyticsManager.cs](#)

4-7. 정직하게 적어야 할 한계

항목	내용
세션 토큰 난수	Cloud Code 런타임에 Node <code>crypto</code> 가 없어 <code>Math.random()</code> 조합. 방어력은 "서버가 만들어 Protected에 저장하고 대조"에서 나오지만 암호학적으로 안전하진 않음
시간 검증 상한	일당 30초 — 실제 플레이보다 훨씬 느슨. 100일 = 50분만 버티면 통과
저장본 위조	점수·저장본 대조를 의도적으로 포기(4-1). 저장본을 위조하는 치터는 통과. "안티치트 완결"이 아니라 "정상 유저 손실 없는 선"
강제 업데이트	서버는 최소 버전 상수만 내려주고 판정은 클라이언트 — 오프라인 재현성을 택한 대신 강제력 없음
욕설 필터	부분문자열 매칭 — 자모 분리·유사문자 (l/1)·특수문자 삽입은 못 막음. 55KB 목록을 소스에 인라인
감사 로그	어드민은 <code>actor_email</code> 을 받지만 <code>logger.info</code> 뿐. 확장 자리만 확보
리더보드 닉네임	metadata 갱신 제약 때문에 조회마다 최대 100명 Cloud Save 병렬 읽기. 1시간 TTL 캐시로 완화했지만 확장성 낮음
코드 중복	Cloud Code Scripts가 모듈 <code>import</code> 를 제한해 뱀 판정 로직이 두 파일에 복제됨
테스트 범위	어드민 사이트 273줄이 이 시스템의 유일한 자동 테스트. Cloud Code JS-C# 매니저는 미커버

5-1. SO ↔ CSV 툴 — "무음 파손"을 막는 diff + 검증

문제 밸런스 데이터를 스프레드시트로 왕복 편집하는데, SO 필드와 CSV 컬럼이 어긋나도 컴파일러는 잡아주지 않습니다. 엉뚱한 컬럼 값이 엉뚱한 필드에 들어가도 조용히 성공합니다 — "빌드는 통과해도 데이터가 무음으로 깨진다(특히 컬럼 인덱스 밀림)."

해결 5파일 2,661줄의 에디터 툴. 세 탭 모두 쓰기 전에 diff를 보여주고 사람이 체크박스로 승인합니다.

탭	동작
Export	SO → <code>_preview.csv</code> 만 생성, 원본 CSV와 diff. 메인 13 + 서브리스트 12 테이블
Applier	preview → 원본 덮어쓰기를 별도 승인 단계로 분리
Import	CSV → SO 결과를 메모리에서 먼저 계산 → diff → 부분 승인 → 의존 순서(Material → ... → Dialogue)로 적용, 체크된 삭제 행은 예셋 삭제

```
// CSVImportTab.cs:560 - 스키마를 규칙 배열로 선언, 오류 1건이면 그 타입 전체 중단
if (Validate("WeaponData", 9, new[]{
    "str", "str", "str", "enum:Grade", "enum:WeaponType", "int", "str", "int", "str"}) > 0) return;
```

- `Validate`: 컬럼 수 · int/float/bool 파싱 · `enum:Grade` 는 13종 enum 맵에서 `Enum.IsDefined`. 오류 행·헤더명·실제값을 찍음
- `CSVDiffCore.Compute`: StaticID 키 합집합 순회 → Added/Removed/Modified, 컬럼별 `"basePrice: 100→120"` 생성. 개행 정규화로 가짜 diff 억제, 서브리스트 행은 `__N dedup`
- 엑셀에 열린 CSV 잠금 감지, diff 로그 파일 저장
- 유지 규약: SO 필드를 건드리면 같은 작업에서 Export/Import/Validate/TypeNames를 함께 고친다 — CLAUDE.md에 체크리스트로 고정

5-2. 다국어 — 프리팹 1,109참조를 건드리지 않고 4개 언어

출발점 문자열 약 2,300개가 전량 하드코딩. 전 UI가 한글 전용 폰트 `Maplestory Light SDF` 하나를 82파일 1,109곳에서 참조 — 가나·한자 글리프가 없어 ja/zh는 렌더링 자체가 불가능했습니다.

해결 ① 폰트 폴백 체인 (코드 약 10줄)

Asset Table로 폰트를 스왑하면 머티리얼 프리셋까지 흔들려 기각. 대신 배경 폰트는 그대로 두고 ja/zh 폰트를 폴백으로 추가, 로케일 변경 시 폴백 순서만 재배열합니다(`ja` → [`JP`, `SC`], `zh` → [`SC`, `JP`]). 이유는 **Han unification** — ja/zh가 공유하는 한자 코드포인트는 폴백 1순위 폰트 자형으로 그려지므로 순서가 곧 자형입니다.

해결 ② 아틀라스 Static/Dynamic — 실측이 최초 계획을 뒤집음

옵션	글리프	용량	판정
실사용 문자만 Static	1,984	~6.4MB	닉네임 등 임의 입력이 두부(□)
ja+zh 상용 합집합 Static	~7,500	~24MB	4배 무거워지고도 인명 한자(詰·翊) 실패
ko Static + ja/zh Dynamic	11,326 + 런타임	TTF 20MB	채택 — 열거 가능한 것(한글 11,172자·게임 텍스트)은 굵고, 불가능한 것(플레이어 입력)은 Dynamic

해결 ③ "빠진 게 없다"를 스크립트로 증명

기존 판정(프리팹 한글 수 - LSE 수)이 완료를 증명하지 못했습니다 — 정규식 `\u[A-C]...` 이 한글 D 대역(평 등)을 통째로 놓쳐 4건 누락. 그래서 `Audit-UiText.ps1` 이 프리팹/씬 YAML 그래프를 훑어 TMP마다 어느 스크립트의 어느 필드가 `.text`를 쓰는지 역추적해 6종 판정(EXCLUDED / ATTACHED / CODE_DRIVEN / REVIEW / UNKNOWN_SCRIPT / MISSING)을 내리고, `Verify-Localization.ps1` 이 9항목(4로케일 키 일치 · 빈 번역 0 · en/ja/zh에 남은 한국어 0 · SO 표시 키 1,002건 전부 해석 ...)을 exit code로 검사합니다. `dotnet build` 7초 컴파일 게이트로 Unity를 열지 않고 일괄 치환을 검증했습니다.

규모: String Table 7컬렉션 **2,110키**(Data 1,002 · UI_Screens 413 · Dialogue 219 · Names 208 · UI_Common 176 · UI_Messages 91 · AppName 1) × 4로케일. 6 Phase 중 5 완료, 법적 문서 로컬라이즈만 남음. ko 데이터 테이블은 만들지 않고 `DataLocalizer`가 ko/미번역이면 SO 원문을 반환해 이중 관리 제거.

코드: `Scripts/Editor/CSVImportTab.cs#L1017 (Validate)` · `CSVDiffCore.cs#L54 (Compute)` · `CSVImportTab.cs#L461 (ApplyToSO)` · `Scripts/Systems/LocalizationManager.cs` · `Scripts/Systems/DataLocalizer.cs` · `Tools/Localization/` · `Docs/오늘도장비대여/Development/다국어_도입전략.md`

§6 에디터 툴과 CI

Unity 메뉴 `Tools > Today's Weapon Rental`. `Scripts/Editor/` 20파일 12,609줄 — 자체 코드의 17%가 개발 도구입니다.

툴	줄	역할
Balance Simulator (4파일)	5,026	\$3. 헤드리스 회차 시물 · 스왑 2종 · 리포트
CSV Tool (5파일)	2,661	\$5-1. SO ↔ CSV Export/Import/Apply, diff + 검증 + 승인
WeeklyQuestGenerator	1,002	레벨디자인 문서(아키타입 지도 · 도입 스케줄 · 조합 규칙)로 주간 의뢰 SO 일괄 생성
AdventureLogExporter	868	모험 로그 5종 CSV. Play 모드는 매니저, 에디터 모드는 <code>gamedata.json</code> 역직렬화
DebugDashboard	702	런타임 상태 인스펙터 — <code>GameData/Inventory/Visitors/Adventures</code> 4탭, 검색·필터, 0.5초 갱신
Adventurer / VisitorNPC Preview	976	외형 데이터 선택 → 스폰 → Idle/걷기/싸우기 모션 확인 (전용 카메라 + <code>RenderTexture</code>)
WeaponEffectDataGenerator · PresetConverter · ProfanityImporter · AndroidApkBuild · ScreenshotCapture	1,309	부가효과 SO 등급별 생성 · 아트메이커 프리셋 → 모험가 변환 · 금치어 импорт · 원클릭 APK · F12 스토어 규격 캡처

GitHub Actions ① `ugs-cloudcode-deploy.yml` — `Tools/CloudCode/**` 변경 시 `deploy.py diff → push → diff`. PR에서는 diff만. concurrency 그룹으로 동시 배포 직렬화.

GitHub Actions ② `analytics-schema-check.yml` — C# 호출부와 `events.json` 대조, 대시보드 문서 render 후 diff 확인. 어긋나면 실패.

§7 출시와 운영

날짜	버전	내용
2026-03-16	—	최초 커밋
2026-07-30	0.0.1 알파	대장간 강화 재료 → 진화 재료, 성공률 상세 패널, 새 레시피
2026-08-03	0.0.2	앱 이름 한국어화, 영어·일본어·중국어 간체 추가 , 대화창 가시성
2026-08-04	0.0.3	저녁 스킵, 순위 업로드 버그, 본인 닉네임 표시. 같은 날 강제 업데이트·계정 삭제·배포 재시도·CI 자동화
2026-08-17	1.0.0 정식	Google Play 정식 출시 (bundle 10, AAB 164.5MB)

플레이테스트 지적 → 시스템으로 대응

지적	대응
"성공률 텍스트가 뭘 의미하는지 모르겠다"	문구 수정이 아니라 클릭 시 상세 패널 (이벤트별 확률 툴팁, 기본 색상 Config) 신설
"저녁에 할 일이 없어 맥이 끊긴다"	기존 <code>SkipMorning()</code> 패턴을 대칭 확장한 <code>SkipEvening()</code> — 저비용 재사용
"앱 이름이 영어다"	한국어화하며 <code>AppName</code> String Table 신설 → 이후 ja/zh 앱명이 얹히는 자리가 됨
"대화창 이름/역할 가시성"	표시 계층 수정

유저는 문구가 아니라 **이해와 리듬**을 지적했고, 대응은 UI 요소가 아니라 시스템 확장이었습니다.

솔직하게 — 남아 있는 약점

항목	사실
자동 테스트 0개	Tests/ 폴더는 있으나 비어 있음. 대신 에디터 툴 diff/검증, PowerShell 감사 9항목, dotnet build 게이트, CI 스키마 대조, 어드민 pytest로 안전망을 깔았지만 게임 로직 회귀 테스트는 없음
테스트 부재가 설계를 막음	문서에 명시 — "성공률 계산기·이벤트 진행기 분리 같은 deepening은 하지 않는다. 자동 테스트 인프라가 없는 상태에서 큰 구조 변경은 후순위"
거대 파일	TutorialManager.cs 2,865줄(partial 미적용, region 21개), AdventureManager.Calculations.cs 1,896줄, SimCore.cs 3,124줄
UI 비중	UI/ 33,607줄로 전체의 45% — View가 "표시만 한다"는 원칙 대비 두꺼움 (View 74 파일, 평균 272줄)
시뮬레이터 이중 관리	수치는 단일 원본이지만 공식은 미래 . 런타임 계산식 변경을 시뮬에 강제하는 장치 없음
명명/배치 불일치	CameraZoomController가 Manager 계층, TraitConfig.asset이 Config 폴더 밖
알면서 안 고친 낭비	메인메뉴 타이틀 글자 11개를 위해 36.4MB 폰트 아틀라스 (재베이크 시 수십 KB)

이 프로젝트에서 배운 것

- **측정할 수 없는 것은 조정할 수 없다.** 시뮬레이터 이전의 밸런싱은 매번 임의 수치였습니다. 도구를 만든 뒤 "지표가 0이면 밸런스 결론 전에 봇부터 의심하라" 같은 함정 체크리스트까지 쌓였고, 결정의 근거가 남기 시작했습니다.
- **클라이언트가 쓰는 값은 판정에 쓰지 않는다.** 서버 시각 스탬프, Protected 키, writeLock 조건부 갱신 — 이 세 가지로 대부분의 위조·경합이 정리됐습니다. 반대로 "정상 유저가 없는 검증"은 빼는 것이 맞다는 것도 배웠습니다.
- **실측이 계획을 뒤집으면 계획을 버린다.** 다국어 아틀라스는 "릴리즈 전 Static 재베이크"가 최초 계획이었지만, 글리프 수를 실제로 세어 보니 4배 무거워지기도 목적을 못 이루는 안이었습니다.
- **안전망은 사고를 맞고 자란다.** 감사 스크립트의 REVIEW/UNKNOWN_SCRIPT 판정, 저장 규약 3가지, 배포 초안 탐지 — 전부 실제 사고 뒤에 추가된 것입니다. 그 경위를 문서에 남겨서 같은 사고를 두 번 맞지 않게 했습니다.
- **다음에 다시 한다면** — 도메인 매니저의 계산 로직을 처음부터 MonoBehaviour 없는 순수 C#으로 분리해 EditMode 테스트와 시뮬레이터가 **같은 코드**를 쓰게 하겠습니다. 미래 이중 관리와 테스트 부재가 같은 뿌리에서 나왔기 때문입니다.

저장소 안내

GitHub 저장소는 유료 에셋·리소스를 제외한 **자체 작성 코드와 문서만** 담은 열람용입니다. 본문의 **파일 #L줄** 표기는 저장소의 같은 경로·같은 줄을 가리킵니다 (원본 파일을 그대로 복사했습니다). 설계 문서 65편과 밸런스 변경 기록 35건, 시뮬레이션 리포트 샘플이 **Docs/**에 있습니다.

<https://github.com/seunghee-2002/Todays-Weapon-Rental-code>